
INF1008 Data Structures and Algorithms

Tutorial 1

Name: Timothy Chia

Student ID: 2501530

1. Write an algorithm (pseudocode is sufficient) that returns the index of the first occurrence of the maximum element in an array $s[0], \dots, s[n-1]$. Use only basic for loops and if statements.

```
arr: list[int] = [3, 2, 1, 5, 3, 2, 1, 5]

def find_largest_index(a: list[int]) -> int:
    max_idx: int = 0 # start with first element as largest.

    # iterate remainder of array, O(n + 1)
    for i in range(1, len(a)):
        if a[i] > a[max_idx]:
            max_idx = i

    return max_idx

idx: int = find_largest_index(arr)

print(f"idx: {idx}") # idx: 3
print(f"arr[{idx}]: {arr[idx]}") # arr[3]: 5
```

2. Write an algorithm (pseudocode is sufficient) that reverses the order of elements in an array $s[0], \dots, s[n-1]$. Use only basic for loops and if statements.

```
# question 2
arr: list[int] = [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

def reverse(a: list[int]) -> list[int]:
    n: int = len(a) // 2

    for i in range(0, n):
        left = i
        right = (len(a) - i) - 1

        a[left], a[right] = a[right], a[left]

    return a

print(reverse(arr)) # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

3. Write an algorithm (pseudocode is sufficient) that outputs the largest and second-largest values in an array $s[0], \dots, s[n-1]$. Assume $n > 1$ and that all elements are distinct. Use only basic for loops and if statements.

```
arr: list[int] = [8, 6, 4, 2, 9, 7, 5, 3, 1]

def find_largest_two(a: list[int]) -> tuple[int, int]:
    first_max_idx: int = 0
    second_max_idx: int = 1

    # edge case
    if a[second_max_idx] > a[first_max_idx]:
        first_max_idx, second_max_idx = second_max_idx, first_max_idx

    for i in range(2, len(a)):
        is_gt_first_max: bool = a[i] > a[first_max_idx]
        is_gt_second_max: bool = a[i] > a[second_max_idx]

        if is_gt_first_max:
            first_max_idx, second_max_idx = i, first_max_idx
        elif is_gt_second_max:
            second_max_idx = i
        else:
            continue

    return a[first_max_idx], a[second_max_idx]

first_largest, second_largest = find_largest_two(arr)

print(f"first_largest: {first_largest}") # first_largest: 9
print(f"second_largest: {second_largest}") # second_largest: 8
```

4. Given a sorted array $s[0], \dots, s[n-1]$ where $n > 1$ and $s[i] \leq s[i+1]$ for all valid i , write an algorithm (pseudocode is sufficient) that inserts a value x such that the array remains sorted. Use only basic for loops and if statements.

```
arr: list[int] = [1, 3, 5, 7, 9, 11, 13, 15]

def insert(a: list[int], x: int) -> list[int]:
    n: int = len(a)

    # iterate list and check if element to insert is
    # lt current element then insert at position
    for i in range(0, n):
        if x < a[i]:
            a.insert(i, x)
            return a

    # element to insert is larger than all current elements
    a.append(x)
    return a

insert(arr, 17)

print(arr) # [1, 3, 5, 7, 9, 11, 13, 15, 17]
```

5. Consider the following algorithm for finding the maximum element in an array:

```
function arrayMax(a, n):
    currentMax = a[0]
    for i = 1 to n
        if a[i] > currentMax
            currentMax = a[i]
    return currentMax
```

Determine how many times the assignment statement `currentMax = a[i]` executes in the best case and in the worst case.

```
# question 5
worst_case: list[int] = [1, 2, 3, 4, 5]
best_case: list[int] = [5, 4, 3, 2, 1]

def max(a: list[int]) -> tuple[int, int]:
    max_idx = 0
    count = 0

    for i in range(1, len(a)):
        if a[i] > a[max_idx]:
            max_idx = i
            count += 1

    return a[max_idx], count

_, count = max(worst_case)
print(f"In the worst case, max_idx = i executed {count} times.")
# In the worst case, max_idx = i executed 4 (n - 1) times.

_, count = max(best_case)
print(f"In the best case, max_idx = i executed {count} times.")
# In the best case, max_idx = i executed 0 times.
```